

SISTEMAS
DISTRIBUIDOS

Práctica 6.

Servicios web.

MICHELLE ANGUIANO RODEA

ESCOM

PROFESOR CHADWICK CARRETO
ARELLANO

GRUPO: 7CM6



Instituto
politécnico
nacional

ESCOM

1. Introducción

En los sistemas distribuidos actuales, los servicios web permiten la comunicación entre diferentes aplicaciones o plataformas a través de Internet. Un servicio web RESTful es una forma moderna y ligera de implementar dicha comunicación, utilizando los principios de la arquitectura REST (Representational State Transfer).

REST se basa en el uso del protocolo HTTP y los verbos estándar como GET, POST, PUT, PATCH y DELETE, los cuales representan las operaciones que se pueden realizar sobre los recursos.

Cada recurso se identifica mediante un URI (Uniform Resource Identifier) y los datos se transmiten en formatos ligeros como JSON o XML, aunque JSON se ha convertido en el estándar más común por su simplicidad.

Este tipo de servicios tiene grandes ventajas:

- Permite la interoperabilidad entre diferentes sistemas, lenguajes y plataformas.
- Usa protocolos y formatos estándar, lo que facilita su desarrollo.
- Es escalable, eficiente y fácil de mantener, ya que cada recurso se gestiona de manera independiente.

En esta práctica se implementó un servicio web RESTful simple en Java, con un servidor que gestiona una colección de libros y un cliente que consume dicho servicio, utilizando la API `java.net.http` incorporada desde Java 11.

2. Problemática

El objetivo de la práctica es implementar y consumir un servicio web RESTful que permita gestionar una colección de libros mediante las operaciones GET y POST.

El servicio debe cumplir los siguientes requerimientos:

1. El cliente podrá realizar una petición GET `/books` para obtener una lista de todos los libros registrados en formato JSON.
2. El cliente podrá enviar una petición POST `/books` para agregar un nuevo libro, enviando los datos (id, categoría y título) en formato JSON.
3. El servidor debe responder con mensajes claros en formato JSON tanto en caso de éxito como de error.
4. La arquitectura debe estar separada en tres componentes principales:
 - Clase común (Book)

- Servidor (BookServer)
- Cliente (BookClient)

El reto consiste en manejar correctamente las solicitudes HTTP, la serialización y deserialización de datos en formato JSON, así como la correcta respuesta del servidor a cada tipo de petición.

3. Solución

Para resolver la problemática, se diseñó una arquitectura cliente-servidor RESTful desarrollada en Java 17.

El servidor se implementó con la clase `HttpServer` incluida en el paquete `com.sun.net.httpserver`, y el cliente con la clase `HttpClient` disponible desde Java 11.

La solución consta de los siguientes componentes:

1. **Book.java** – Define la estructura de los objetos libro con atributos id, categoría y título.
2. **BookServer.java** – Implementa un servidor REST que responde a las solicitudes:
 - GET /books: devuelve una lista de libros en formato JSON.
 - POST /books: agrega un nuevo libro a la colección a partir de un JSON recibido.
3. **BookClient.java** – Actúa como cliente HTTP que consume el servicio REST, realizando peticiones GET y POST y mostrando las respuestas.

El intercambio de información entre cliente y servidor se realiza en formato JSON, garantizando compatibilidad e independencia de plataforma.

4. Implementación

Clase Común (Book.java)

Define la estructura de los libros manejados por el servicio:

```

Book.java > ...
1  // Book.java
2  public class Book {
3      private int id;
4      private String categoria;
5      private String titulo;
6
7      public Book(int id, String categoria, String titulo) {
8          this.id = id;
9          this.categoria = categoria;
10         this.titulo = titulo;
11     }
12
13     public int getId() { return id; }
14     public String getCategoria() { return categoria; }
15     public String getTitulo() { return titulo; }
16
17     public void setId(int id) { this.id = id; }
18     public void setCategoria(String categoria) { this.categoria = categoria; }
19     public void setTitulo(String titulo) { this.titulo = titulo; }
20
21     @Override
22     public String toString() {
23         return String.format(format: "{ \"id\":%d, \"categoria\": \"%s\", \"titulo\": \"%s\" }", id, categoria, titulo);
24     }
25 }
26

```

Servidor (BookServer.java)

Implementa el servicio RESTful con los métodos GET y POST:

```

import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import java.io.*;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.stream.Collectors;

public class BookServer {

    private static final List<Book> books = new ArrayList<>();

    Run | Debug
    public static void main(String[] args) throws IOException {
        int port = 8080;
        HttpServer server = HttpServer.create(new InetSocketAddress(port), backlog:0);
        System.out.println("Servidor iniciado en http://localhost:" + port);

        // GET /books → Lista todos Los Libros
        server.createContext(path: "/books", new HttpHandler() {
            @Override
            public void handle(HttpExchange exchange) throws IOException {
                String method = exchange.getRequestMethod();
                switch (method) {
                    case "GET":
                        handleGet(exchange);
                        break;
                    case "POST":

```

```

        handlePost(exchange);
        break;
    default:
        handleUnsupported(exchange);
    }
}

});

server.setExecutor(executor:null); // usa el default
server.start();
}

private static void handleGet(HttpExchange exchange) throws IOException {
    String response = books.stream()
        .map(Book::toString)
        .collect(Collectors.joining(delimiter:",", prefix:"[", suffix:"]"));
    sendResponse(exchange, status:200, response);
}

private static void handlePost(HttpExchange exchange) throws IOException {
    InputStream is = exchange.getRequestBody();
    String json = new String(is.readAllBytes(), StandardCharsets.UTF_8);
    // parse simple JSON (sin librerías externas)
    try {
        Map<String, String> map = parseJson(json);
    }
}

```

```

        Book newBook = new Book(
            Integer.parseInt(map.get(key:"id")),
            map.get(key:"categoria"),
            map.get(key:"titulo"));
        books.add(newBook);
        sendResponse(exchange, status:201, body:"{\"status\":\"Libro agregado\"}");
    } catch (Exception e) {
        sendResponse(exchange, status:400, body:"{\"error\":\"JSON inválido\"}");
    }
}

private static void handleUnsupported(HttpExchange exchange) throws IOException {
    sendResponse(exchange, status:405, body:"{\"error\":\"Método no permitido\"}");
}

private static void sendResponse(HttpExchange ex, int status, String body) throws IOException {
    ex.getResponseHeaders().set(key:"Content-Type", value:"application/json; charset=UTF-8");
    byte[] bytes = body.getBytes(StandardCharsets.UTF_8);
    ex.sendResponseHeaders(status, bytes.length);
    try (OutputStream os = ex.getResponseBody()) {
        os.write(bytes);
    }
}

private static Map<String, String> parseJson(String json) {
    json = json.trim().replaceAll(regex:"[{}\\"]", replacement:"");
    Map<String, String> map = new HashMap<>();
}

```

```

        for (String pair : json.split(regex:",")) {
            String[] kv = pair.split(regex:"=");
            if (kv.length == 2) map.put(kv[0].trim(), kv[1].trim());
        }
        return map;
    }
}

```

5. Resultados

Al ejecutar el servidor y luego el cliente, se observaron los siguientes resultados:

```
PS C:\Users\miaro\Documents\Sistemas distribuidos\Practica 6> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\miaro\AppData\Roaming\Code\User\workspaceStorage\3e297357a20527c56198dc4dd279e27e\redhat.java\jdt_ws\Practica 6_c2637521\bin' 'BookServer'  
Servidor iniciado en http://localhost:8080
```

```
PS C:\Users\miaro\Documents\Sistemas distribuidos\Practica 6> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\miaro\AppData\Roaming\Code\User\workspaceStorage\3e297357a20527c56198dc4dd279e27e\redhat.java\jdt_ws\Practica 6_c2637521\bin' 'BookClient'  
Respuesta GET:  
[]  
  
Respuesta POST:  
{ "status": "Libro agregado" }  
  
Lista de libros actualizada:  
[{"id":1,"categoria":"Programación","titulo":"Sistemas Distribuidos"}]  
PS C:\Users\miaro\Documents\Sistemas distribuidos\Practica 6>
```

Esto demuestra que:

- El servidor maneja correctamente las peticiones GET y POST.
- Los datos son enviados y recibidos en formato JSON.
- El cliente puede agregar libros y luego obtener la lista actualizada.
- La comunicación cliente-servidor se realiza exitosamente a través del protocolo HTTP.

6. Conclusión

La práctica permitió comprender el funcionamiento de los **servicios web RESTful**, así como el proceso de **consumir y exponer APIs HTTP** mediante peticiones GET y POST.

Se concluye que:

1. RESTful es una arquitectura ligera, eficiente y ampliamente utilizada para la comunicación entre sistemas distribuidos.
2. Java ofrece herramientas integradas (HttpServer y HttpClient) que facilitan la creación y consumo de servicios web sin dependencias externas.
3. El intercambio de datos en formato JSON simplifica la interoperabilidad entre diferentes tecnologías.

4. Esta práctica sienta las bases para desarrollar APIs más complejas, seguras y escalables en entornos distribuidos.

7. Referencias

- Ceupe. (2022). *Servicios*. Recuperado de:
<https://www.ceupe.com/blog/servicio.html>
- IONOS Digital Guide. (2021). *Web Services: definición y ejemplo*.
- AWS Documentation. (s.f.). *¿Qué es REST?*
- Brave Developer. (2023). *Qué es REST, RESTful, API RESTful y JSON*.
- Oracle Docs. (2024). *Java HTTP Client API Overview*.